Computer Graphics: Lecture 15

Lighting part 1: Diffuse (Lambertian) Lighting

Welcome back!

Last time we learned about real, hierarchical scene management.

We made a simple recursive scene-based renderer.

This was important: recursive scene nodes are the core of most 3D graphics engines.

There's something we're still missing though, and it's very important for realism in our scenes…

...Lights!

Lighting is critical for human perception of an object.

Without it, the only other real option is texturing, but it's just not enough on its own.

In this lecture, we're going to take steps to implement the lighting system you see in this screenshot.

How does light work?

We probably learned the basics in science class:

- Light is emitted as photons from light sources, such as the sun.
- The photons get reflected, refracted, or absorbed as they move.
- If one of them happens to bounce into our eye, there's a chance one of our eye's rods or cones will pick it up, and aggregate it with the other photons happen to be hitting at around that time.
- Then we *see* the light.

So, seems to suggest a simple algorithm: track the photons as they leave the light source. [Why not?]

The problem

There are a *lot* of photons, and most of them don't hit our eyes.

As a result, simulating those would be entirely wasted time for the purpose of computer graphics.

However, what if we went in reverse?

That is, instead of tracking each lightbeam as it bounces around, what if we shot beams the other direction, out of our camera, and then checked if they hit a light source?

That works

That algorithm works; it's called **raytracing**.

Raytracing fundamentally works like this:

- For each pixel in the screen, fire a ray out into the world
- If it intersects a triangle, determine which light-sources are visible from the point of intersection.
- Based on the distances to those light-sources, we can add the color contributions from each one, and set the color of the pixel.

That's it. It's actualy a very simple algorithm. In fact, people have made raytracers that fit on the back of a business card.

So why aren't we making raytracers all the time?

Raytracing is a great algorithm. It's elegant, and it has the capacity to produce photo-realistic graphics. Unfortunately, it's still pretty slow.

There's a reason why RTX was such a big deal: real-time raytracing was kind of the holy grail of the computer graphics space for a while.

Even now, it's still rather limited in its effects.

What's so slow about raytracing?

The issue with raytracing

There are two dominant approaches to 3D graphics:

- Rasterization (what we're doing)
- Raytracing

Rasterization means "take this list of triangles, and draw their pixels to the screen."

Raytracing means "take this list of pixels and draw them according to which triangles they intersect".

The algorithms seem similar. However, modern renderers, even if RTX is supported, will do most of their drawing work with rasterization.

The issue with raytracing (2)

The issue with raytracing is that *testing whether a ray collided with one of the millions of triangles in your scene is slow.*

Let's do some basic math. If you're in 2160p resolution with 16:9 aspect ratio, sometimes known misleadingly as 4K, there will be 3840 by 2160 pixels. Roughly 8 million.

Each of those needs to be tested somehow against the triangles in the scene, which can also number in the millions.

Yes, there are all kinds of optimizations we can use to reduce the number of intersection tests, but the fact remains that the problem is hard.

The issue with raytracing (3)

When we draw a triangle with rasterization, we don't have to check whether it was collided with, we just draw the triangle.

Basically, "here are the bounds of the triangle, fill it with the pixel colors that get returned from the fragment shader."

There are still a large number of pixels, to fill in, but if a triangle is off screen, it will get clipped and we won't fill its pixels in.

As a result, instead of being in the order of number-of-pixels times number-of-triangles, it ends up being in the order of number-of-pixels plus number-of-triangles (assuming a non-degenerate case, like drawing a million triangles in order from far to near)

So why bother?

And yet, RTX seems to be here to stay, and raytracing doesn't seem to be going anywhere. So why? What's the big deal?

The main reason is this: raytracing enables global illumination without requiring "hacks".[1]

What is global illumination?

[1]There are other techniques besides raytracing that enable global illumination, such as Radiosity.

Global illumination

The lighting technique that we're going to learn today is a **local illumination** technique.

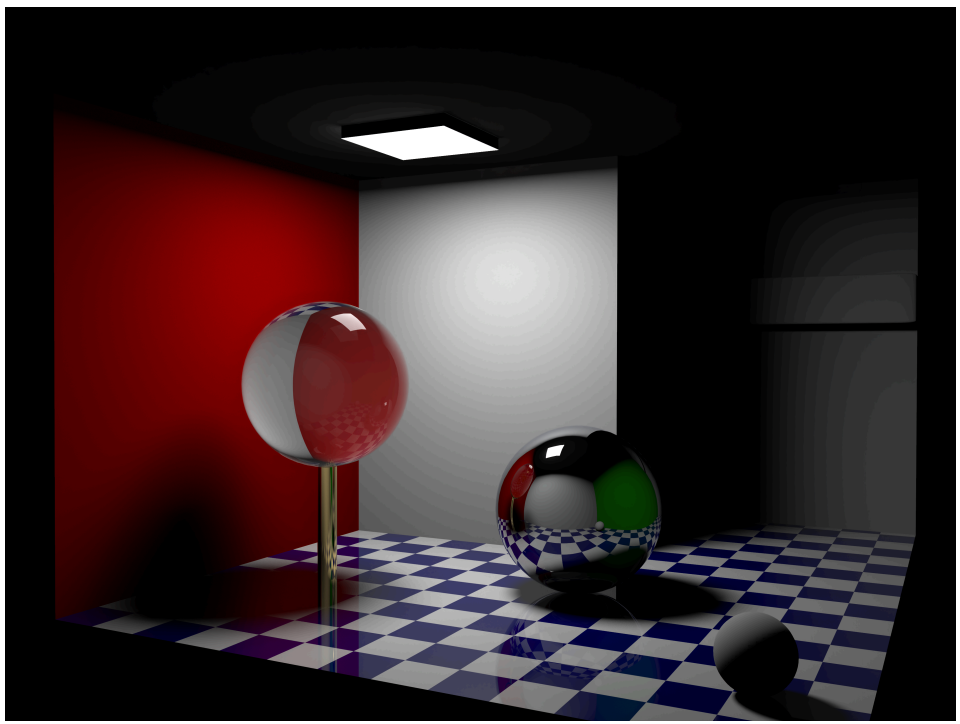
It works to help define what objects are in the scene and provide some basic realism, but it will never be perceived as photo-realistic.

The main issue is that we will *only be considering the impact of lights on the colors of our surfaces.*

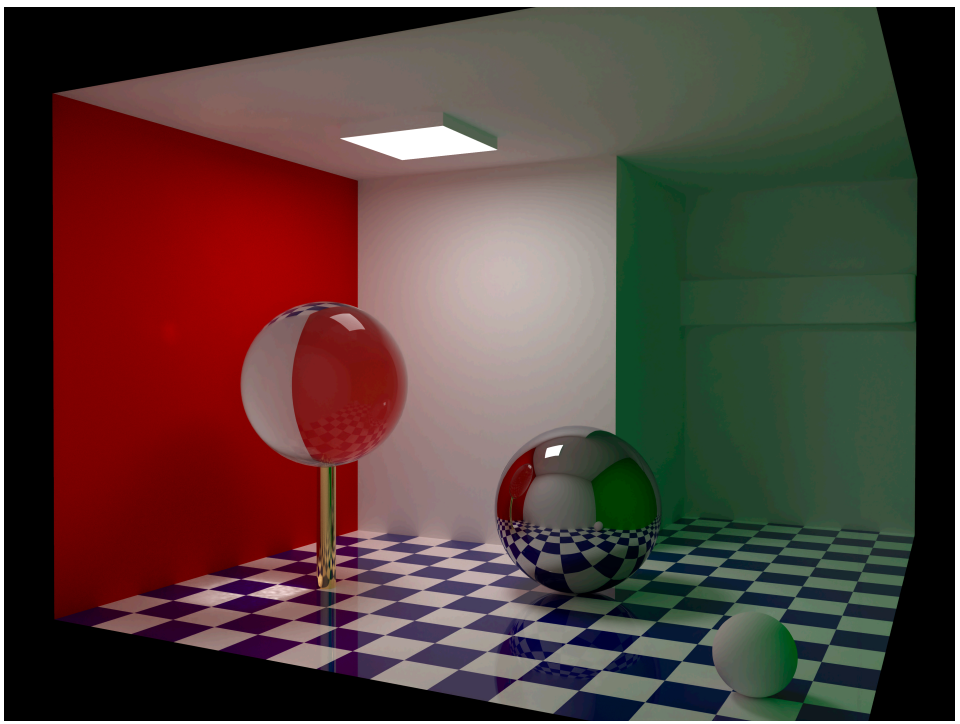
Only? What's the other option?

Well, when a light is cast upon a surface, some of it will be reflected. *The surface ends up becoming a light source.*

Global illumination vs local illumination



Local illumination only



Global illumination

Things to notice about the comparison

In both images, the light at the top is the primary source of light.

In the left image, that is the *only* source of light. As a result, it looks dark. The back wall is in shadow; it's not in direct view of the light.

In the right image, light is conserved. The walls end up being their own light source. It affects how we draw the objects: notice that the ball is green-ish on its right side. From that, we can infer that there is a green wall to the right that we can't see. We also see the green color cast on the white wall in the back, confirming this.

Global illumination is required for photorealism. It's also very slow.

Why global illumination is slow

Think back to our discussion on raytracing: where an “anti-photon” is sent out from the camera to see which triangle it hits.

We said that when an intersection is detected, we then check to see how illuminated that point is by a light source.

In global illumination, *everything is a light source.*

As a result, we need to send out more rays to see what *those* hit. And those raycasts could potentially require more rays. It can become an exponential problem if we don’t set a threshold for how much light we care about or how many ray-tests we will allow.

What to do instead

Even if we were to do global illumination, we still want to rasterize as much as we can get away with.

Light is additive, so if we draw a triangle with a good *base color*, we can add highlights onto it later by testing out some rays from the surface.

Ultimately, it becomes a game of identifying how to account for *most* of the light. If a triangle is directly under 4 light sources, we can probably sum those up and call it a day. It's only when the triangle is off in the corner and mainly illuminated by reflected light that we see a big benefit from ray tracing.

Starting from rasterization

This discussion has all been rather theoretical, as WebGPU doesn't support RTX or AMD's ProRender natively.

And you're definitely not getting realtime performance without hardware acceleration of ray intersection testing.

I just wanted you to understand the space, and know the difference between global and local illumination.

With that, let's take quesitons and start learning a much faster, local-illumination system...

Questions?

Basic lighting

There is an equation that fully describes the amount of light leaving a point in space along a viewing direction. It is called the rendering equation. Here it is:

$$L_{o(x,\omega_o,\lambda,t)} = L_{e(x,w_o,\lambda,t)} + L_{r(x,\omega_o,\lambda,t)}$$

$$L_{r(x,\omega_o,\lambda,t)} = \int_{\Omega} f_{r(x,\omega_i,\omega_o,\lambda,t)} L_{i(x,\omega_i,\lambda,t)} (\omega_i \cdot n) dw_i$$

So just plug and chug right?

Of course not. Most of those terms are only relevant under global-illumination. We can use a much simpler model.

Lambertian reflectance

The model we will learn in this lecture is called Lambertian Reflectance.

It is named after Johann Heinrich Lambert, and described in his book Photometria. The book contained, among other things, a bunch of experiments that were designed to learn how much light gets reflected in a variety of configurations.

This is before photography, so the book has visual setup guides, such as this one, depicting candles being placed in a correct configuration.

Despite the complicated setup, the book presents a remarkably simple rule for determining how much light is scattered from a surface, which we will learn soon.

Reflectance

There are several ways that light can interact with a surface:

- It can be absorbed by it.
- It can be "bent" (refracted)
- It can be reflected in a straight line. This is what happens to surfaces that have relatively few imperfections. This is **specular** reflection.
- It can be scattered in all directions. This is **diffuse** reflection.

The more chaotic a surface's microscopic texture, the more it will scatter and absorb light, and the less it will be reflected with a clear highlight.

Diffuse light

A surface that scatters light perfectly randomly in all directions is called a **lambertian** surface.

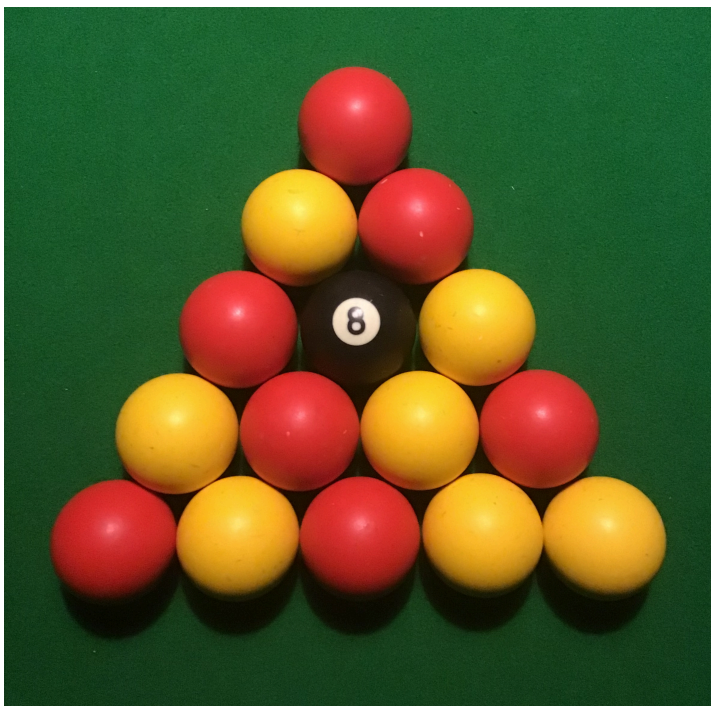
Perfectly lambertian surfaces don't really exist, but some real world surfaces come pretty close:

- Wood
- Unpolished rocks
- Frayed cloth

Basically, things that are rough and not shiny. These surfaces scatter light **diffusely**. Light that is scattered is **diffuse light**. Light that is reflected coherently without scattering is **specular light**.

Diffuse vs. specular light

These pool balls have specular highlights on them. They are smooth, not-very-diffuse surfaces. Image By SMcCandlish - Own work, CC BY-SA 4.0, link



These pool balls are a lot rougher. Their specular highlights are barely visible. They are more diffuse. Image By SouthcottC - Own work, CC BY-SA 4.0, link

Modelling diffuse light

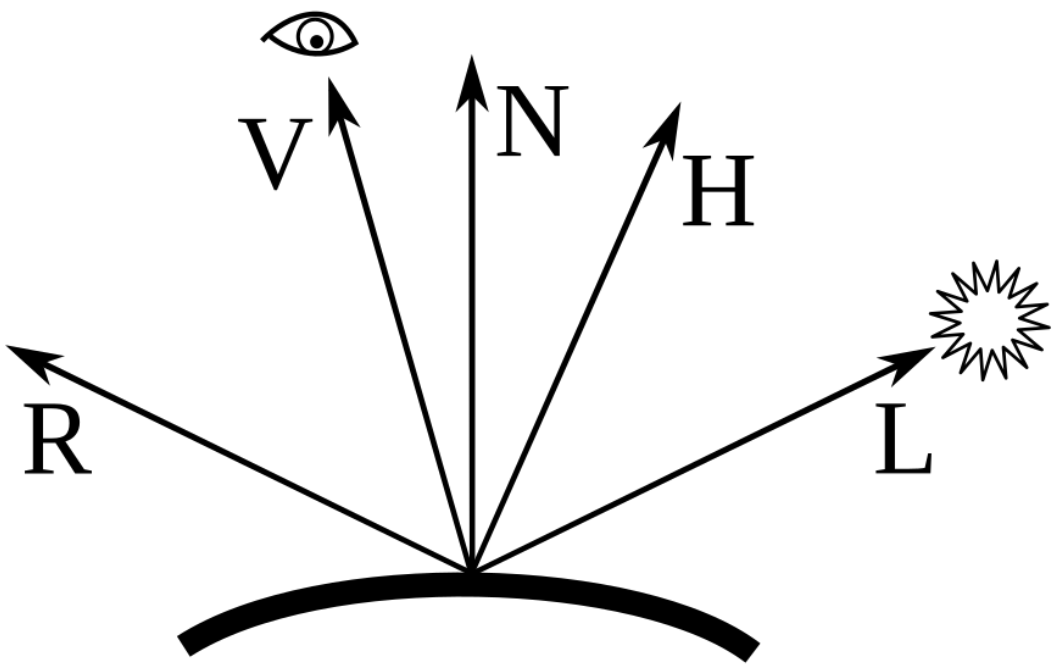
It turns out, when a surface is very diffuse, it is easy to calculate how bright it should be, and this is one of the things Lambert discovered.

Fundamentally, the amount of light the surface will receive is based on the angle between the surface's *normal* and the direction to the light.

The normal is the unit-length vector pointing exactly perpindicular away from the surface at a point.

And the direction to the light is a vector pointing from the point on the surface, in the direction of the light.

Modelling diffuse light (2)



Only consider `N` and `L` for now. `N` is the normal vector. `L` is the vector pointing to the light.

Modelling diffuse light (3)

It turns out, when the light is scattered in all directions, the brightness depends entirely on the cosine of the angle between `N` and `L`.

That is the surface is brightest when the surface is exactly pointing towards the light source, and gets dimmer as the surface rotates away, becoming 0 when it is perpendicular to or facing away from the light.

[How can we compute this quickly?]

Modelling diffuse light (4)

It's just the dot product:

```
let cos_angle = dot(normal, light_dir)
let diffuse_brightness = max(cos_angle, 0);
```

Remember: the dot product is just the magnitudes of the two vectors times the cosine between them.

In this case, the magnitudes are 1, so it ends up being just cosine.

Cosine is maximized at 1 when two vectors are exactly in line. That's when the surface scatters *all* the light, and will therefore be brightest. As it turns parallel, it scatters less and less, and gets dimmer.

Questions?

So where is the normal?

This calculation has to happen in one of the shaders, but that means the shader needs the normal.

Normals are normally something we associate with faces. A triangle, for example, has a normal defined by the cross product between two of its edges.

However, there is no “face shader” that we have access to in WebGPU. We need normals to be associated with vertices.

Normal attributes

So now it's finally time to add another attribute. Recall that attributes are pieces of information we associate with vertices.

Previously we saw:

- Position, which is basically required.
- Color
- UV coordinates

Now we are going to add the normal vector, representing a vector that is perpendicular to the surface at the vertex.

Computing the normal

In reality, if you build a 3D model in a program like Blender, it will compute and export the normals for you.

However, when normals are exported in OBJ files, they become annoying to process. Therefore, it's easier to compute them at runtime.

To compute normals, there is a simple algorithm that works well:

- For each vertex: store which faces it touches from the index list.
- For each face: compute the cross product.
- For each vertex again: add together all the cross products from each face it touches, and then normalize the result.

Computing the normal (2)

The logic behind this is that we want to approximate the normal at a point, but we only know the normal for sure in the regions around the point.

We don't want to treat them equally, otherwise we would end up with artifacts between lots of tiny triangles and large open faces.

Therefore, we weight them by area. This happens naturally as we take the cross product: the cross product is the product of the magnitudes of the two input vectors, so as the triangle gets larger the cross product is larger.

Computing the normal (3)

When we normalize the sum of cross products, we end up with the weighted average.

That is, suppose our vertex is touching 3 triangles: two with a cross product of length 1, and one with a cross product of length 4.

We take the sum of those three vertices. We get a vertex pointing roughly in between the three, but it's 6 units long. The length 4 one will contribute more to its total length in each dimension.

Therefore, when it gets normalized, it will still point more towards the direction of the big triangle.

Computing the normal code

```
const vertHasFaces: number[][] = new Array(this.nVerts);
const faceHasNormal: vec3[] =
  new Array(Math.floor(indices.length / 3));
```

We built two data structures: one which maps a vertex index to which faces it touches, and another which maps each face to a normal.

With both of these, we can then iterate over each vertex to get its faces, and compute the weighted normal from them.

Note: we're going to start by iterating over the *indices* rather than the vertices. Since every 3 indices defines a face, we can chunk our indices into groups of 3, and add the face to each of those indices.

Computing faces

```
for (let i = 0; i < indices.length; i += 3) {
    const faceNo = Math.floor(i / 3);
    vertHasFaces[indices[i]!] ??= [];
    vertHasFaces[indices[i]!]!.push(faceNo);
    vertHasFaces[indices[i + 1]!] ??= [];
    vertHasFaces[indices[i + 1]!]!.push(faceNo);
    vertHasFaces[indices[i + 2]!] ??= [];
    vertHasFaces[indices[i + 2]!]!.push(faceNo);
    ...
```

We iterate in groups of 3. The face number is therefore the index number divided by 3.

For each index in the face, we check if it has had any faces defined yet. If not, `vertHasFaces[indices[i]!]` will be undefined…

The ??= operator

The `??=` operator is similar to `??`, and has the same relationship to it that `+=` has to `+`.

That is, it checks if the thing on the left is undefined or null, and if it is, it assigns the thing on the right. In this case the empty array, `[]`.

Once we're sure that there's an array corresponding to that index, we add the face number to it. We do the same thing for all three indices in the face.

When this process is finished, we know exactly which faces each vertex is touching. Our numbering, starting from face 0, is arbitrary, but consistent.

Computing the normal code (2)

```
const a = positions[indices[i]!]!;
const b = positions[indices[i + 1]!]!;
const c = positions[indices[i + 2]!]!;
const d = vec3.create();
const e = vec3.create();
const n = vec3.create();
vec3.sub(d, b, a);
vec3.sub(e, c, a);
vec3.cross(n, d, e);
faceHasNormal[faceNo] = n;
```

Here, for the face, we get its 3 vertices (`a`, `b`, `c`) and compute two edge vectors for them (`d` and `e`). We compute the cross product, `n`, from that, and store it in our data structure.

Computing the normal code (3)

```
this.vertData = [];
for (let i = 0; i < this.nVerts; i++) {
  const n = vec3.create();
  for (let face of vertHasFaces[i]!) {
      vec3.add(n, n, faceHasNormal[face]!);
  }
  vec3.normalize(n, n);
  this.vertData.push(...positions[i]!, ...n);
}
```

Then, for each vertex, we add all the face normals together into `n`, normalize it, and store it inside the vertex adata, after the positions.

The `...` operator means "expand the collection into this other collection". `push(...v)` is like `push(v[0], v[1], v[2])`. It's a shortcut.

Wiring up the shader

Now that we have position *and* normal data in our vertex, we need to be sure that both attributes go to the shader.

That means our pipeline will need at least two attributes defined in the `vertex` section, and our shader module will need at least two inputs in the vertex shader.

See `sample12` if you're still a little shaky on how to do this, but I strongly recommend trying it first so that you can remember it better.

But there's one more thing that we need…

The normal matrix

We currently have a model matrix, that represents how we want the 3D model we're drawing to be positioned in the scene.

When we multiply the vertex by the model matrix, we get a new vertex in world-space.

We can use the model matrix to rotate, scale, or translate the model within the scene.

But now we have this extra piece of information coming along for the ride. What should happen to the normal vector when we rotate, scale, or translate the model?

The normal matrix (2)

Well, it depends:

- When we *rotate* the model, the normals should rotate with it
- When we *translate* the model, nothing should happen to the normals, because they are vectors and vectors do not have a position (i.e., if we used homogenous coordinates, normals would have a `w` of 0).
- When we *scale* the model, the normals will need to scale by an inverse factor. That is, doubling the x means halfing the x of the normals.

The last one is the tricky one. If we're just translating or rotating, we could use the model matrix for normals. They would ignore translation, and be rotated correctly. Unfortunately, scaling messes things up.

The normal matrix (3)

Why should nothing happen when we scale?

Because if you make a triangle bigger, the normal stays the same.

Even if you make it bigger in one direction (i.e., stretch it).

If we applied a scale matrix that doubled the x component, it would skew all the normals in the `x` or `-x` direction, and mess up the lighting.

Technically, if the scale is uniform, we could just renormalize after the scale stretched or shrunk it, but since we want to be able to scale in a non-uniform way if we choose, let's learn how to properly transform normals.

The normal matrix (4)

This explanation largely comes from the WebGPU unleashed chapter I asked you to read last time.

Here is how we figure out how to construct a matrix that will correctly transform the normal vector.

First, consider that if v is a *tangent vector* (meaning, a vector that is *parallel* to a surface), then the dot product between it and the normal vector (n) should be zero: $n \cdot v = 0$

We can rewrite this using matrix multiplication if we transpose n:
$n^T \times v = 0$

The normal matrix (5)

We plan to multiply every vertex of the mesh by the model marix. That means that any tangent vector will have that matrix applied to it. So if the model is rotated, the tangent vector is rotated, etc.

However, we want to preserve the relationship that the dot between the tangent vector and the normal equals zero.

Therefore, if we multiply the tangent by matrix `M`, we can multiply the normal by the inverse of that matrix: $n^T \times M^{-1} \times M \times v = 0$

This is allowed because $M^{-1} \times M = I$, so we haven't actually changed anything.

The normal matrix: what that result means

That result is saying “if you want the normal to *stay* normal, you have to post-multiply it by that inverse matrix”

If we don’t do that, we’ll lose the *normalness* after we transform the vertex.

But wait, that seems weird…

The normal matrix (7)

This implies that, as long as we multiply the normal by the inverse of the model matrix, we're fine. But wait, that doesn't make sense...

That would mean that if we rotate the model by 30 degrees in one direction, we rotate the normal by 30 degrees in the opposite direction!

The reason for this seeming contradiction, is that the normal is transposed, and multiplied on the other side. This flips the rotation order again.

A pure rotation matrix will, when transposed and flipped, rotate in the same direction as the original.

The normal matrix (8)

But wait, we don't have a transposed normal vector. We have a regular normal vector. It will have the same orientation as our position vector.

So we need to tranpose again:

$$n^T \times M^{-1} = \left(\left(n^T \times M^{-1} \right)^T \right)^T = \left(M^{-T} \times n \right)^T$$

Two important things:

1. M^{-T} means "inverse transpose". Either invert M and tranpose it or transpose M and invert that. The result is the same.
2. The result of this is that the last quantity will, when multiplied by the transformed vertex, be 0. However, we don't need the last transpose. That transpose is only for the dot product we will not be computing.

The normal matrix (9)

Therefore, if we take the inverse tranpose of the model matrix, and multiply *that* by the normal, it will preserve the property that the dot product between the normal and a vector parallel to the plane will be zero.

That *doesn't* mean that the result will actually be a normal vector: there are two requirements for a normal vector!

1. That it be perpendicular to the plane
2. That it have unit length

We have only guaranteed the first one!

The normal matrix (10)

Therefore, once we have our normal matrix, which was derived from the model matrix, we can multiply our normal by it *and then we re-normalize the normal.*

```
@vertex fn vs(
    @location(0) pos: vec3f,
    @location(1) normal: vec3f,
) -> VertexOutput
{
    var vo: VertexOutput;
    vo.world_pos = m_model * vec4f(pos, 1);
    vo.pos = m_viewProj * vo.world_pos;
    vo.norm = normalize(m_normal * normal);
    return vo; // world_pos is non-view-transformed position. We're not
}  // using it in this shader, but it's useful for goureaud shading later.
```

A quick gut check about normal matrices

Let's try to understand why multiplying by the inverse transpose of the model matrix works for normals:

- If the model matrix is a rotation matrix, the transpose of it is the same as the inverse (this is a property of rotation matrices). Therefore, the inverse tranpose does the same thing as the original matrix (we're inverting twice)
- If the model matrix is a translation matrix, we can easily ignore the translation component by cutting out the translation part (the normal matrix is usually a 3-by-3 matrix.
- If the model matrix is a scale matrix, the transpose is the same as the original. Therefore, we end up *only* inverting, which is what we want. A stretch by 2 in the x direction needs to squeeze the normal by (one-half) [we can whiteboard this if everyone wants, it's easier to understand interactively]

Getting the normal matrix

We could take the model matrix, clear its w-column, invert it, and transpose it, but normal matrices are so common that `gl-matrix` will do it for us.

The operation is `mat3.normalFromMat4`:

```
mat3.normalFromMat4(this.matNormal, this.matModel);
```

Notice: the output is a 3-by-3 matrix. We treat normals as 3-dimension vectors because they are *always* vectors. They don't have that w-component that points have. We don't want to translate normals.

Vector alignment

Unfortunately, this last part is tricky, and if I had realized it before I started building this course, I would probably have gone with a matrix library that targets WebGPU specifically.

We can't just send a `gl-matrix` 3-by-3 matrix to the video card.

The reason is *alignment*.

Vector alignment

Alignment is important for fast memory access, CPU or GPU.

We don't want data to be misaligned.

That is, we want to avoid this kind of situation:

```
var f: f32; // single float, bytes 0-4
var v1: vec4f; // bytes 5-20
```

The reason is that for reasons of size and cost, the underlying electronics both CPU and GPU vector units are built to assume that copies will only happen from certain addresses

Vector alignment (2)

That is, the underlying vector unit can copy data to and from bytes 0-15 and to and from bytes 16-31, and so on, in 16-byte chunks.

The underlying electronics are wired this way. Making the vectors byte addressable would increase the cost.

If we allowed a float to bump the vector over by 4-bytes, it would be out of alignment. Then, to access it, we would need to access both neighboring memory blocks and do bit-operations on the results to piece together the vector.

This would be dramatically slower.

Vector alignment (3)

To avoid this slowdown, WebGPU will insert empty bytes, called padding, in order to guarantee alignment.

So the float will be in the same place, but the vector will be located at byte 16 instead of 4. There will be 12-bytes of padding inserted between them.

Now both the float and the vector can be addressed in one memory operation, without needing multiple accesses and shifts.

Why does this matter?

Vector alignment (4)

Because matrices are stored as basis vectors.

Specifically, a 4-by-4 matrix is stored as 4, 4-dimensional vectors.

But what about a 3-by-3 matrix? Like a normal matrix?

It's stored as 3, **4-dimensional** vectors. That's not a typo. It's 48 bytes.

Why? because a 3-float vector would not be aligned to 16-bytes.

Unfortunately, OpenGL did *not* work this way. OpenGL assumed that a 3-by-3 matrix was stored as 9 floats that would then be unpacked by the driver into the correct alignment.

Vector alignment (5)

Therefore, when we send a 3x3 gl-matrix matrix to the video card, it will not get the correct values. The x-component of the second basis will be lost (it will be put in the padding), the y component will become the x, and so on down the line. It will be completely wrong.

So, we have to unpack it ourselves:

```
const n = this.matNormal;
const unpacked = [
  n[0], n[1], n[2], 0, // the zeros are the padding
  n[3], n[4], n[5], 0, // we make sure that the xyz components go in the
  n[6], n[7], n[8], 0  // first 3 components of each vector in the GPU.
];
this.device.queue.writeBuffer(this.matNormalBuf, 0, new f32a(unpacked));
```

Vector alignment (6)

Alternatively, we could have just made the normal matrix a 4-by-4.

Then the alignment works out naturally.

Just remember: if you're copying a matrix to the video card and it's not 4-by-4, you need to be careful.

Questions?

Ambient light

Of course, we can't completely ignore global illumination. Our simple lighting code can fake it by adding a fixed light color to every fragment.

This fixed color is called **ambient light**.

It lets us, e.g., give ever object in a room a base level of light to simulate the light that would otherwise be bouncing around.

It's not a substitute for global illumination, but without it, the scene would be too dark.

Adding light

Light is *additive*. If I shine a red and a blue light at a white wall, it will reflect back purple.

That means, if we have multiple sources of light, we can just add their brightnesses together.

Brightness isn't stored separately, it's a factor of the light's color. If you want a dimmer light, you can divide its color vector.

(you can also change its attenuation, which is how quickly its brightness falls off over time. We'll discuss this next time)

We use linear space to add light. Gamma correctness is important here!

Materials

One last thing: different materials will scatter or absorb different amounts of light.

We'll learn more about this next time, but for now, we can define how much diffuse or ambient light a surface reflects as its material. This way, surfaces that absorb light can be dimmer.

Materials can be different per channel: some materials will absorb more red, some more blue, etc. Since materials can be unique to an object, they're in the same bindgroup as the model and normal matrix.

Okay, that's theory for now, let's see the rest of the shader...

The fragment shader

```
@fragment fn fs(vo: VertexOutput) -> @location(0) vec4f {
    let diffuse_term =
        max(dot(dir_light_dir, normalize(vo.norm)), 0) *
        dir_light_color *
        mtl_diffuse;
    let ambient_term = ambient_color * mtl_ambient;
    let color = diffuse_term + ambient_term;

    return vec4f(color, 1.0);
}
```

Notice: we *re-normalize* the normal. Why? Because it will be linearly interpolated. That does not preserve distance. [whiteboard]

The fragment shader (2)

The color of the fragment is just the diffuse light it receives plus the ambient light it receives.

The ambient term is the amount of ambient light we're pretending is bouncing around, times the amount of ambient light the material reflects.

The diffuse term is the dot product between the light direction and the surface normal, bounded to be 0 at the minimum. We also multiply the diffuse reflectance of the material.

The sample

I really went all out with the sample. It lets you tune all of these things independently.

You can change the color of the light, the material settings, and the ambient state. You can even change the mesh in a little drop down box.

This sample is a bit more complicated than usual, feel free to ask questions. Let's spend some time exploring it.

Next time

I left out a super important thing: attentuation

This sample pretended that the only thing that mattered was where the light was. What about how far away it is? There are different kinds of light! (we're modelling a directional light source, like the sun)

Also, what about reflective, metallic surfaces? The aren't diffuse! We need a more complex model to handle specular surfaces.

Next time we cover the Phong model and goureaud shading. This is where we finally get up to about the mid-90s in terms of our technological advancement.

Questions?